



Become a Solution Architect

Enhance your Technical and Behavioural Skills

What's your
plan for
Sunday?



TECHNICAL TRACK

AGENTIC AI EVALUATION & OBSERVABILITY

MEASURE • MONITOR • UNDERSTAND • IMPROVE



WEEK 5

● LIVE

BEHAVIOURAL TRACK

VISUAL FACILITATION & REAL-TIME ARCHITECTURE

SIMPLIFY • ALIGN • DESIGN • DECIDE



Week 5 - Agenda

Topic	Duration	Presenter
Technical Topic Agentic AI Observability and Evaluation	50 Mins	Ashish & Shorena
Behavioural Topic Visual Facilitation & Real-Time Architecture	30 Mins	Jeff & Krishna
Workshop Walkthrough From Prototype to Production with AWS - Agentic AI Evaluation and Observability	30 Mins	Robert

Additional Learning Resources



8,500+

Titles & Courses

30 Days

Full Access

\$0

Cost To You

Registration link has been shared in Post-Session Email and on Discord



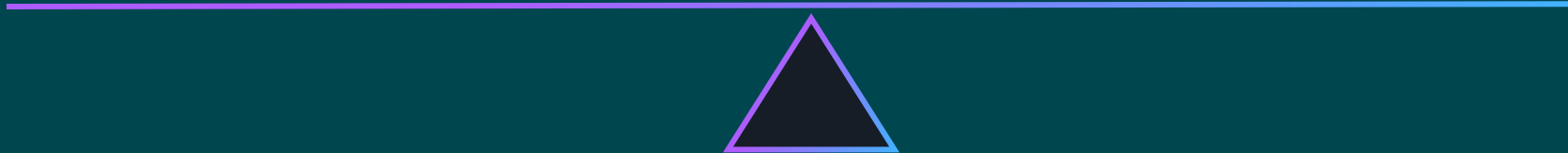
Agentic AI Observability and Evaluation

The Trust Gap

TOP NOTCH
TECHNICAL
CAPABILITY



BETTER
CUSTOMER
EXPERIENCE





Limited visibility

Hard to see what's ahead



Hidden risks

Obstacles may be unseen



Uncertain path

Not sure if you're on the right route



Unknown outcome

Will you reach the destination safely?

 **Fog Lights**
Improved visibility in foggy conditions

 **Nearby Vehicles**
Detects and shows surrounding vehicles

 **Collision Detection**
Monitors distance and alerts to potential collisions

 **Digital Road Signs**
Provides real-time road information

 **Speed Limit**
Clearly visible digital speed limit

 **Drive Safe
Reduced Visibility**

 **Fog Lights**
Improved visibility in foggy conditions


48 m


32 m


60

 **Driver Assistance Indicators**
Shows active safety features

12°C 60 km/h 8:24

 **GPS / Navigation**
Shows your route and upcoming turns

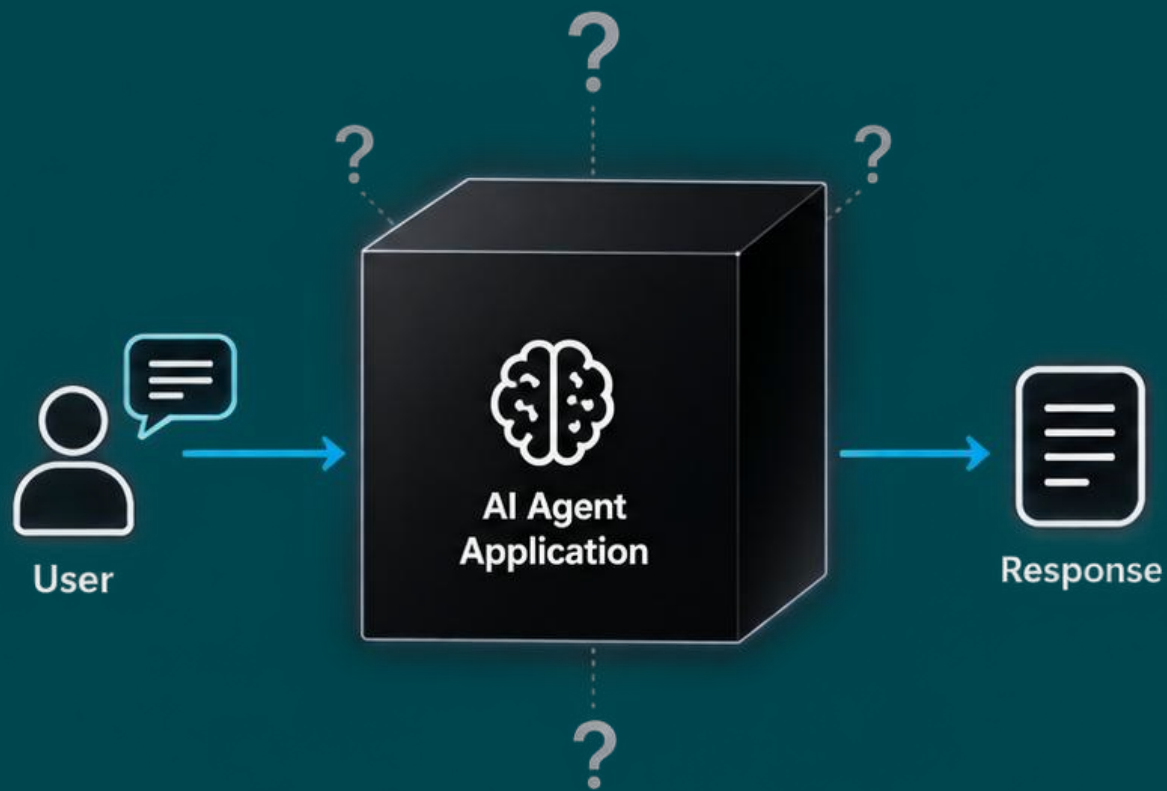
12 km Turn right 8:24 23 min 18 km

 **Surround View**
Detects vehicles in adjacent lanes and blind spots

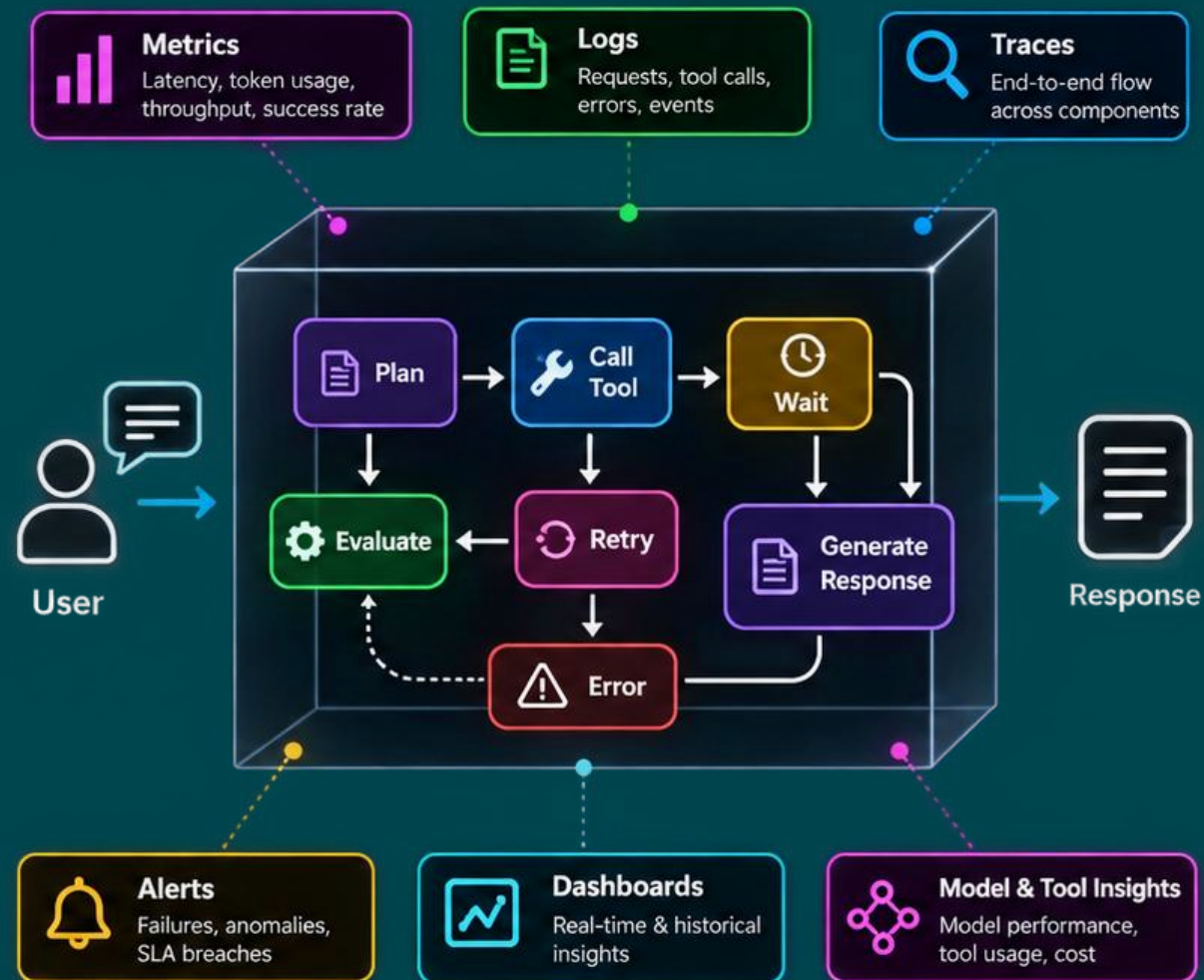
 **GPS / Navigation**
Shows your route and upcoming turns

 **Surround View**
Detects vehicles in adjacent lanes and blind spots

AI Agent Application (No Visibility)



AI Agent Application (With Observability)



Monitoring vs Observability

Monitoring

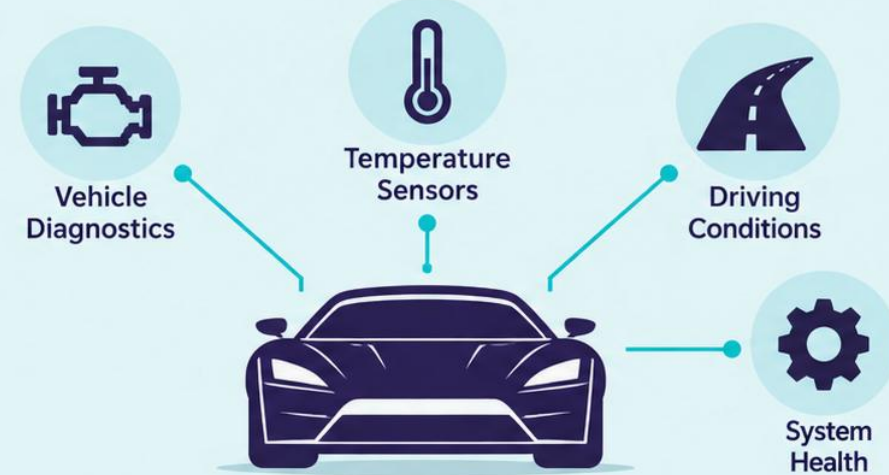
Dashboard warning light comes on



What it tells you:
Something is wrong.

Observability

Use dashboard readings + diagnostics + operating conditions



What it helps you do:
Understand why it is happening.

Monitoring Shows the Problem – **Observability** Helps Explain Why

Monitoring vs Observability: Technical Difference

Aspect	Monitoring 	Observability 
Primary goal	Detect known issues and track system health	Understand why the system is behaving a certain way
Typical question	“Is something wrong?”	“Why is it happening?”
Data used	Mainly predefined metrics, dashboards, and alerts	Correlates metrics, logs, traces, events, and system context
Best suited for	Threshold breaches and expected failure conditions	Complex, unexpected, or multi-layer problems
Example	Alert when GPU utilization drops or temperature rises	Correlate GPU, network, storage, and workload signals to identify the root cause

Unique Challenges for Agentic AI Observability

- 1 Agents follow non-deterministic control flows powered by model's reasoning
- 2 Root cause analysis focuses on complex sequence of chained calls
- 3 Collecting and analyzing of genAI invocations
- 4 Assessing system health holistically for operational performance and quality

Points of Failure for an Agent

Quality Failures

- Hallucinations & factual errors
- Faulty reasoning & planning
- Poor tool selection
- Inconsistent outputs

Cascades to

Reliability Issues

- Context loss
- Poor handling of errors
- Security gaps

Results in

Inefficiency

- High cost
- Latency
- Loss of Customer Trust



Three Pillars of Observability



Metrics

Numerical
measurements
collected
over time



Logs

Timestamped
records of
individual
events



Traces

End-to-end
records of how
a request flows
through the system

Better Visibility

| Faster Troubleshooting

| More Reliable Applications

Observability for Agentic AI

What it captures?

What can you learn?

Metrics



Sessions, invocations, latency, token usage, errors

Why has agent latency increased by 15%?

Logs



Agent events, requests/responses, tool activity, errors

What error occurred during agent or tool execution?

Traces



Agent steps, model calls, tool calls, dependencies and timing

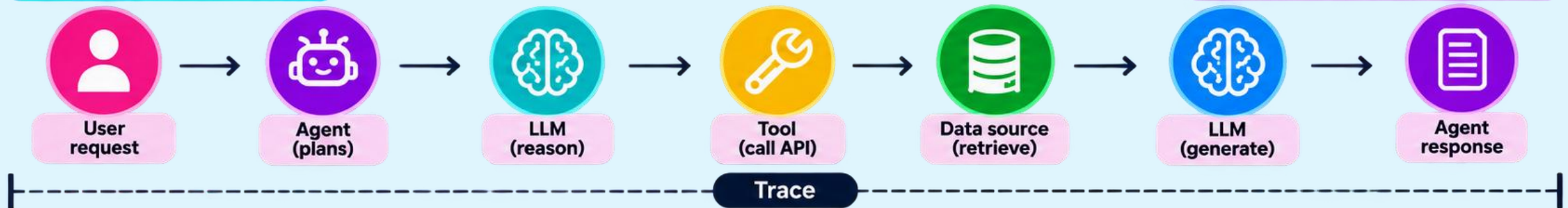
What path did the agent take, and where was time spent?

Traces – Tracking an AI Agent Request

Food Delivery Analogy



Agentic AI Execution



A trace is the complete record of a single agent execution/request, showing the sequence of operations performed from start to finish.

What is Instrumentation?

- Instrumentation adds observability code to your application so it can collect telemetry (data about what is happening) during execution.

Agent Application → Instrumentation → Traces

Stage	How it is done?	AWS implementation
1. Instrument	Add code or agents that emit telemetry	Agent application + ADOT (AWS distribution of OpenTelemetry)
2. Collect / Export	SDK exports data; a collector is optional	Direct OTLP export to CloudWatch, or via ADOT Collector / CloudWatch agent
3. Observability backend	Stores and indexes telemetry	CloudWatch (metrics, logs) + X-Ray (traces), with Transaction Search enabled
4. Analyze / Visualize	Dashboards, alerts, trace exploration	CloudWatch dashboards, Application Signals, GenAI Observability

OpenTelemetry (OTel) is an open-source, vendor-neutral standard and toolkit (APIs, SDKs and a Collector) for generating and exporting traces, metrics and logs.

`</>` Python

Simplified illustration, not runnable code

```
tracer = get_tracer("my-agent")
```

```
def call_llm(prompt):
```

```
    with tracer.start_span("llm_call") as span:
```

```
        span.set_attribute("model", "my-model")
```

```
        response = llm.invoke(prompt)
```

```
        span.set_attribute("tokens.used", response.token_count)
```

```
    return response
```

AgentCore Runtime auto-instruments CLI-deployed agents; add ADOT for custom spans.

Agentic AI Observability on AWS

Amazon Bedrock AgentCore Observability

Agent-specific
observability for
sessions, traces, spans,
tool execution,
performance and
errors.

Amazon CloudWatch GenAI Observability

Central dashboards
and views for GenAI
resources, agents,
traces, logs, metrics
and troubleshooting.

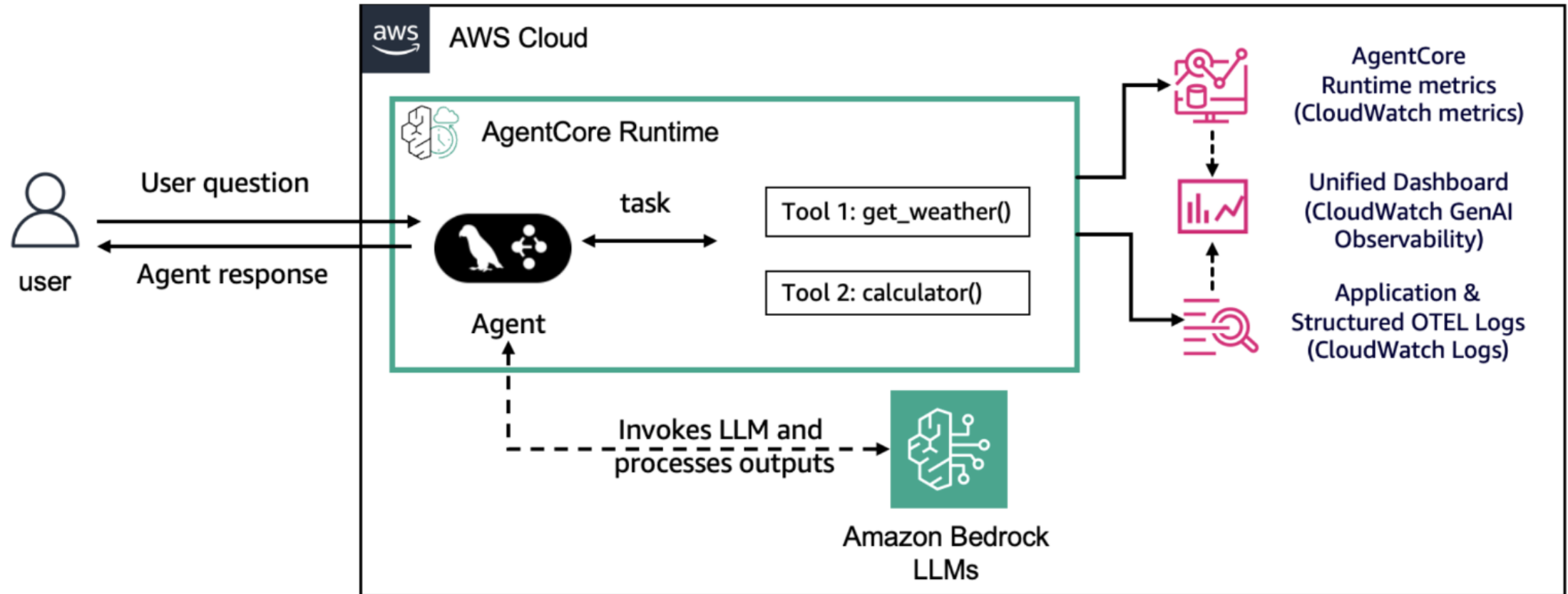
Third Party Observability Tools



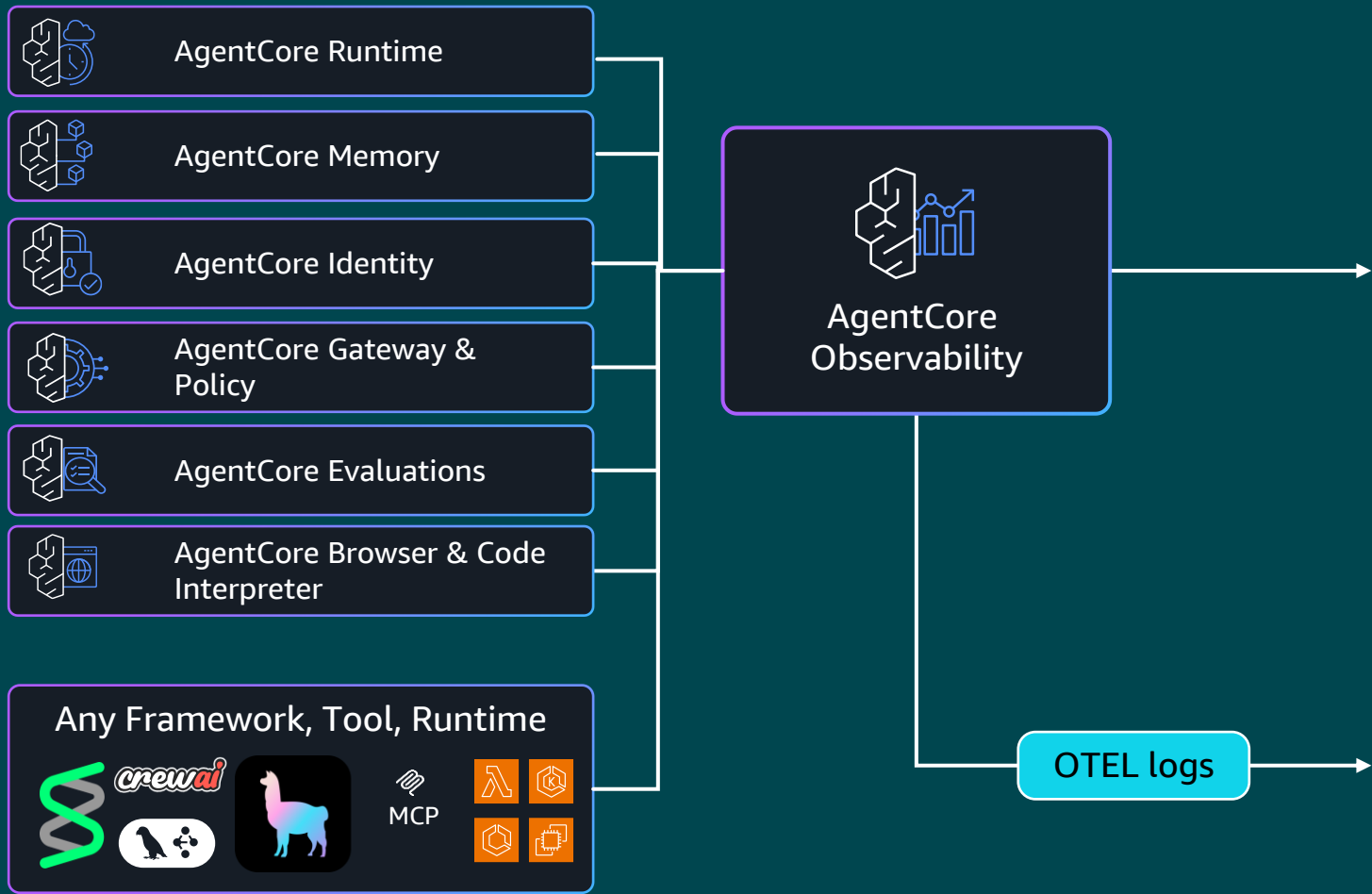
And many more...

And many more...

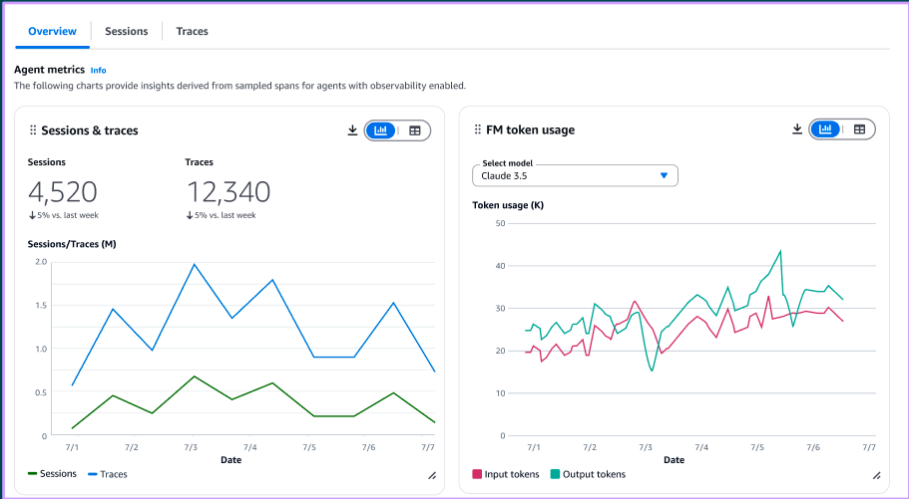
Amazon Bedrock AgentCore Observability



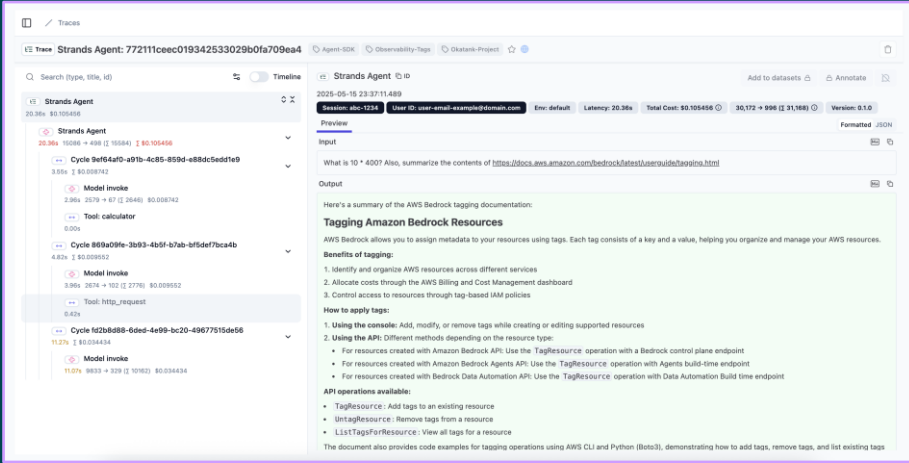
Amazon Bedrock AgentCore Observability



AgentCore Observability dashboards



Third-party observability dashboards





Evaluation

Seeing the Journey Is Only Half the Story



Fog Lights

Improved visibility in foggy conditions



Nearby Vehicles

Detects and shows surrounding vehicles



Collision Detection

Monitors distance and alerts to potential collisions



Digital Road Signs

Provides real-time road information



Drive Safe
Reduced Visibility



Speed Limit

Clearly visible digital speed limit



Fog Lights

Improved visibility in foggy conditions



Driver Assistance Indicators

Shows active safety features



GPS / Navigation

Shows your route and upcoming turns



Surround View

Detects vehicles in adjacent lanes and blind spots

Visibility ≠ Success



- We can now see what is happening. But how do we know the journey is actually good?
- The driving aids give us **visibility**, but visibility alone doesn't tell us whether the driver:
 - Took the **right route**
 - Followed the **traffic rules**
 - Made the **right decisions**
 - Reached the **correct destination**
 - Did so **safely and efficiently**
- **We need evaluation**

AI Agent Requires Two Perspectives

Observability	Evaluation
What is happening?	Is it happening correctly?
See the journey	Judge the journey
Calls, waits, loops, errors	Correct actions and decisions
Latency, failures, tool activity	Goal achievement and quality
Visibility	Quality

How do we know if the agent is doing a good job?



Agent Evaluation

- Did the agent achieve its goal?
- Are your agents making the right decisions?
- Are they using the best tool for the job?
- Are their answers accurate?



Amazon Bedrock AgentCore Evaluations

Fully managed, continuous
quality assessment for AI agents

- Analyze agent behavior for specific quality criteria
- Built-in and custom evaluators
- Results integrate into AgentCore Observability for unified monitoring
- AWS Handles:
 - Scaling & Capacity
 - Rate Limits & Throttling
 - Infrastructure Ops

When to Evaluate?

- Periodically and Continuously

**Update a
model**

**New version
of agent**

**Modify a source
prompt**

**Add and remove
tools**

Key Concepts

Session

A complete user interaction – spans multiple turns and can include multiple traces

Trace

One end-to-end agent request, from input to final response

Span

A single logical operation within a trace: model call, tool invocation, retrieval

Sub-Span

Nested detail within a span — intermediate steps, retries, sub-agent calls

Session: AI Agent Chat

Trace: "Help me track my order"

Span: Parse intent

Span: Generate response

LLM call

Tool call - API

Database call

Final response

Span: Send response

Trace: "Change address"

Span: Validate

Span: Process change

LLM call

Tool call - Auth

Database call

Span: Confirm

Session End

AgentCore Evaluation Metrics

Session level metrics

Goal Success Rate

Did the session
achieve its objective?

Trace level metrics

Trace Level

Correctness
— Accurate output?

Helpfulness
— Useful response?

Faithfulness
— Grounded in facts?

Span (tool) level metrics

Tool Selection
— *Right tool chosen?*

Parameter Accuracy
— *Correct parameters
used?*

13 Built-in AgentCore Evaluation Metrics

Session level metrics

Goal success rate

Trace level metrics

Correctness

Faithfulness

Helpfulness

Response relevance

Conciseness

Coherence

Instruction following

Refusal

Harmfulness

Stereotyping

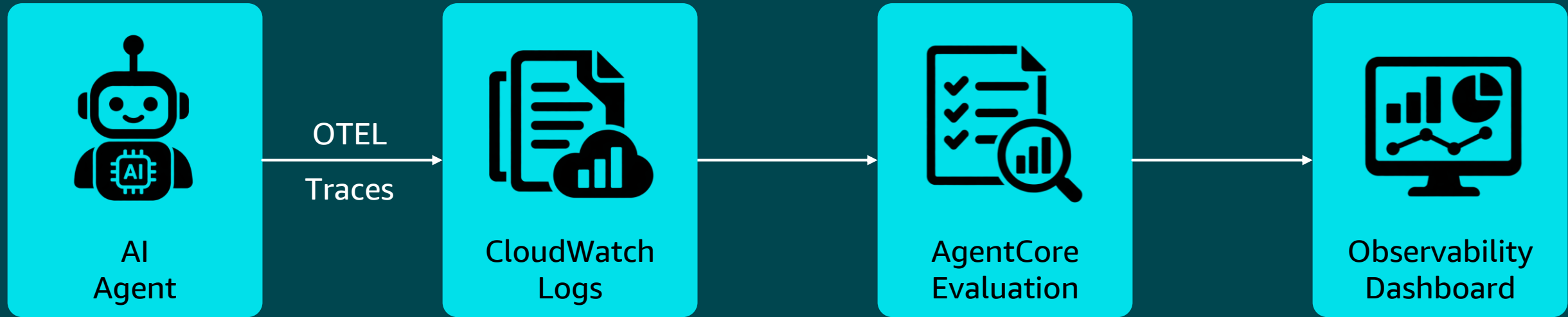
Span (tool) level metrics

Tool selection accuracy

Parameter selection accuracy

Custom metric

How it Works?



Custom Evaluators using LLM-as-a-Judge



Step 1:

Define your criteria

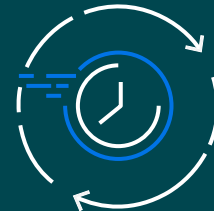
Example: "Purchase intent detection"



Step 2:

Write Evaluation prompt

- Analyze the conversation and determine clear intent to make a purchase
- Consider explicit statement, question about pricing/checkout, and urgency signals
- Scoring rubric:
 - 0.0: No purchase intent
 - 0.5: Browsing with mild interest
 - 1.0: Clear purchase intent expressed
- Provide step by step reasoning before scoring



Step 3:

Select evaluation model



Step 4:

Configure sampling

Best Practices for Observability & Evaluation

1

Start Simple, Then Expand

2

Integrate Early

3

Use Consistent Naming

4

Filter Sensitive Data

5

Configure for Development Stage

6

Review Regularly

7

Framework Agnostic

Minimum Viable Observability

- If you can only do 3 things, start here

1

Instrument traces (OpenTelemetry), costs and performance metrics

— *See what your agent or LLM pp is doing and measure the performance and costs*

2

Define 3-5 custom business-aligned metrics

— *Measure what matters*

3

Run automated evaluation weekly

— *Catch drift early*

Start here. Expand as you mature.



Visual Facilitation & Real-Time Architecture



Hands-on Workshop

Walkthrough

Jupyter Notebook Skills

Watch this 6 minutes YouTube video for getting started.

- <https://youtu.be/suAkMeWJ1yE?si=V7XSR-nWNnnvcQGh>

Walk through basic navigation mentioned in the official Jupyter notebook tutorial by VS Code.

- <https://code.visualstudio.com/docs/datascience/jupyter-notebooks>

Next Steps: Your Turn to Act

1



Complete the
Knowledge
Check

2



Practice the
Hands-on
Workshop

3



Be active and
Support
Your Peers

4



Share Your
Learning on
Social Media